

Technical Project Approach

1.PROJECT: Audio Frame Identifier

Given: Video link , dialogue

To find: Time stamp for the first occurrence of the dialogue and the frame

Method 1 : Download video -> Extract the audio (.wav) -> Transcribe entire audio to obtain words with their timestamp -> Find the first occurrence of the dialogue using sliding window -> Similarity matching score (if greater than threshold) -> Return start time stamp of first word -> Find the frame at that timestamp -> return result

Method 2: Download video -> Extract audio (.wav) -> Load saved transcript/checkpoint -> Search previously processed words -> Transcribe next audio chunk -> Add newly transcribed words to incremental matcher -> Detect candidate when first dialogue word matches -> Extend candidate with subsequent words -> Calculate SequenceMatcher similarity -> If similarity $\geq$ threshold, return first qualifying occurrence -> Otherwise discard candidate when it becomes too long -> Save transcription checkpoint -> Continue with next chunk -> Return start timestamp of first word -> Find frame at timestamp -> Return result

2.LIBRARIES,MODELS

Component	Choice	Why
Video download	yt-dlp (invoked as a subprocess)	Handles a wide range of hosts; resumable, retry-friendly
Audio extraction	ffmpeg	Decodes any container to WAV in one pass
Media inspection	ffprobe	Duration and FPS without decoding the whole file
Speech-to-text	faster-whisper — WhisperModel	Word-level timestamps, CPU-friendly
Model	small.en, device="cpu", compute_type="int8"	Best accuracy/speed trade-off for English on CPU; int8 keeps memory low

Component	Choice	Why
Matching	difflib.SequenceMatcher (stdlib)	No extra dependency; tolerant of small transcription errors
Tests	pytest	integration marker separates tests needing FFmpeg/models

3.APPROACH 1

#	Step	Function
1	Download video (or reuse)	download_video()
2	Extract 16 kHz mono WAV	extract_audio()
3	Load model once	AudioTranscriber.__init__()
4	Transcribe entire audio	AudioTranscriber.transcribe()
5	Flatten segments → words	AudioTranscriber._extract_words()
6	Persist transcript	save_word_timestamps()
7	Slide a window, score each	find_first_dialogue()
8	Extract frame at result.start	extract_frame()
9	Format and return	format_timestamp()

APPROACH 2:

#	Step	Function
1	Download / reuse video	download_video()
2	Extract WAV	extract_audio()
3	Load checkpoint → (words, transcribed_until, complete)	load_progress()
4	Search already-transcribed words first	find_first_dialogue()

#	Step	Function
5	If hit → return immediately (zero transcription)	
6	If complete and no hit → dialogue absent, return None	
7	Read duration once	get_media_duration()
8	Transcribe [transcribed_until, +30 s]	transcribe_next_chunk() → transcribe_segment()
9	Merge new words, keep chronological order	merge_words()
10	Save checkpoint before searching	save_progress()
11	Search accumulated transcript	find_first_dialogue()
12	Hit → stop. Miss and not at end → loop to step 8	—
13	Extract frame at result.start	extract_frame()

4.DIALOGUE MATCHING:

- **Similarity ≥ 0.80** — token-level ratio, so a single misheard word in a five-word phrase still passes. Tolerates Whisper's ordinary errors without matching unrelated speech.

5.DESIGN DECISIONS:

Decision	Reason
Download registry keyed by SHA-256 of the normalized URL	Same video is never downloaded twice; fragments (#t=) stripped so variants of one URL collapse to one entry
Transcript persisted to JSON	Transcription dominates runtime; a second dialogue query on the same video is near-instant
Model loaded once and injected	WhisperModel init is expensive; Method 2 would otherwise reload it every chunk
save_progress() called every chunk	A crash or interrupt costs at most 30 s of work

Decision	Reason
frame_number = round(timestamp * fps)	Uses the stream's actual avg_frame_rate, not an assumed 30 fps

6.UNIT TESTS:

File	Covers
test_matching.py	Normalization, threshold behaviour, first-occurrence-wins , no-match returns None
test_progressive_search.py	Stops early on a hit (via a FakeTranscriber — no model needed); reuses a saved transcript
test_progressive_transcription.py	Chronological merge; empty state for a fresh checkpoint
test_transcript.py	Save/load round-trip; missing file raises
test_transcription.py	Word-level timestamps returned and time-ordered
test_audio.py	WAV produced; missing video rejected
test_frame_extraction.py	FPS, duration, image written, negative timestamp rejected